Task Info

Learning Phase - Task 1

Introduction

The goal of this task is to explore the evolution of modern NLP systems and understand how transformer architectures changed sequence modeling.

To explore different aspects of transformer architectures, the assignment has been divided into three subtasks:

- encoder-only transformers,
- sequence-to-sequence transformers,
- and decoder-only transformers.

The overall focus of the assignment is experimentation, analysis, and understanding architectural tradeoffs rather than simply optimizing benchmark metrics.

Subtask Structure

The assignment is divided into three subtasks, each focusing on a different transformer paradigm:

- Encoder-only transformers
- Sequence-to-sequence transformers
- Decoder-only transformers

Each subtask is further divided into two components:

1. Base Requirements

The base requirements are intended to ensure that everyone:

- implements the core architectures,
- understands the basic workflow,
- and gains exposure to the corresponding transformer paradigm.

All cores are expected to complete the base requirements of all three subtasks.

2. Ablations & Deep Exploration

In addition to the base requirements, each core is expected to pick one subtask of their choice and explore it more deeply through:

- architectural experimentation,
- ablation studies,
- analysis of model behavior,
- visualization,
- and deeper experimentation.

This component carries significant weightage and is intended to encourage:

- experimentation,
- curiosity,
- and deeper understanding of sequence modeling systems.

The focus here is not simply on training larger models, but on understanding:

- why certain architectural choices work,
- where models fail,
- and how different design decisions affect behavior and performance.

LLM Usage Policy

While the use of Large Language Models (LLMs) for assistance is not prohibited, **excessive reliance on them is strongly discouraged**. The objective of this phase is to build your own intuition for architectural tradeoffs and sequence modeling. Outsourcing the heavy lifting to an AI entirely defeats the purpose of the exercise. Do not shortchange your own learning.

If you do utilize an LLM, you must explicitly disclose it. Clearly mention exactly where and how the LLM was used, either through direct inline comments in your code or detailed explanations within your README/report.

Submission Format

A common organization will be created for all cores on github.

Each core is expected to:

1. Create a repository of their own. This will serve as the common point of submission.
2. Inside that, create a folder named `Task-1`

3. Inside `Task-1`, create separate subfolders for each subtask and organize all submissions accordingly.

Example structure:

```
Your_Repository/
|── Task-1/
|   |── Subtask-1/
│   |── Subtask-2/
│   |── Subtask-3/
```

Each subtask folder should ideally contain:

- source code,
- notebooks(with demo/results/plots),
- README/report,
- generated outputs/results,
- and any additional resources relevant to the task.

Deadline

Deadline: 5th June, 2026

Submissions are expected to be pushed to the repository before the deadline.

1. Array Element Ranking

Subtask 1 - Encoder-Only Transformers for Relational Reasoning

Overview

In this task, you will explore how encoder-only transformers process and reason over structured numerical data. You will build and analyze a BERT-style architecture for solving an **Array Element Ranking** problem.

Unlike autoregressive models that process tokens sequentially, encoder-only transformers view the entire sequence simultaneously using bidirectional self-attention. This makes them particularly effective for tasks requiring global relational understanding.

Through this assignment, you will investigate:

- how self-attention performs implicit pairwise comparisons across a sequence,
- how numerical information can be represented inside transformer architectures,
- how positional encodings affect relational reasoning,
- how attention differs from recurrence for global context tasks,
- and how transformers behave under out-of-distribution inputs.

The primary objective is not only to obtain good performance, but also to understand *why* the model behaves the way it does.

Problem Statement

You are given a sequence of integers:

[45, 12, 99, 31]

The task is to predict the **relative sorted rank** of each element.

Sorted sequence:

[12, 31, 45, 99]

Corresponding ranks:

[2, 0, 3, 1]

Meaning:

- 12 is the smallest → rank 0
- 31 is second smallest → rank 1
- 45 is third smallest → rank 2
- 99 is largest → rank 3

The model must output the rank of every token in the input sequence.

This is fundamentally a **global relational reasoning task**, since determining the rank of a number requires comparing it against all other numbers in the sequence.

Dataset

You will work with a synthetic dataset available here.

Suggested Workflow

The following workflow is only a guideline. You are encouraged to go beyond it.

---

1. Build Baselines

Before implementing transformers, start with simpler architectures to understand why ranking is difficult for traditional sequence models.

Possible baselines include:

- Multi-Layer Perceptrons (MLPs)
- RNNs
- LSTMs

Suggested Questions

- How quickly do these models converge?
- Do sequential models struggle with global comparisons?
- Does performance degrade with longer sequences?
- Can the models generalize to unseen patterns?

Compare

- Training convergence
- Validation accuracy
- Stability
- Generalization behavior
- Computational efficiency

---

2. Implement an Encoder-Only Transformer

Implement and train a transformer encoder from scratch for the ranking task.

Treat the problem as a **token classification problem**, where each token predicts its own rank.

Your implementation should include:

- Bidirectional self-attention
- Positional encodings
- Multi-head attention
- Feed-forward networks
- Residual connections
- Layer normalization
- Classification head for rank prediction

Important Note

You may use low-level PyTorch or TensorFlow primitives.

However, the attention mechanism and transformer blocks should ideally be implemented manually instead of importing high-level transformer libraries. The goal is to understand:

- tensor shapes,
- Q/K/V projections,
- attention score computation,

- masking behavior,
- and multi-head aggregation.

---

3. Inference, Visualization & Analysis

Transformers are highly sensitive to data representation.

After training:

Attention Visualization

Extract and visualize attention weights using heatmaps.

Analyze questions such as:

- How does a token attend to larger or smaller numbers?
- Does the model compare elements pairwise?
- Do different heads specialize differently?

Example:

- Does token 45 strongly attend to token 99 while determining its rank?
- Does attention become more structured after training?

Out-of-Distribution Testing

Evaluate the model on sequences that differ significantly from the training distribution.

Examples:

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

[1000, 900, 800, 700, ...]

[5, 5, 5, 5, 5]

Analyze:

- robustness,
- failure modes,

- attention behavior,
- and generalization capability.

Ablations & Experiments

A major component of this assignment is experimentation and analysis.

You are expected to conduct meaningful ablation studies regarding how transformers process non-linguistic numerical data.

A. Data Representation

This is one of the most important aspects of the assignment.

Categorical Embeddings vs Continuous Representations

Investigate:

- embedding layers,
- learned projections,
- normalized numerical inputs,
- raw float inputs.

Questions to explore:

- Does the model treat 5 and 6 as mathematically related?
- Or does it treat them as unrelated symbols?

Sequence Normalization

Experiment with:

- Z-score normalization,
- min-max scaling,
- sequence-wise normalization.

Analyze how normalization affects:

- training stability,
- convergence,
- out-of-distribution performance.

B. Architecture & Attention

RNN/LSTM vs Transformer

Compare:

- sequential processing,
- long-range reasoning,
- parallelism,
- global context understanding.

Positional Encoding Ablation

Remove positional encodings entirely.

Questions:

- Can the model still rank numbers?
- Does order matter for this task?
- What information is lost?

Attention Head Specialization

If using multiple attention heads:

- visualize all heads,
- compare their patterns,
- analyze specialization.

Do some heads focus on:

- large numbers?
- nearby tokens?
- pairwise comparisons?

Depth Experiments

Compare:

- 1-layer encoders
- 2-layer encoders
- 4-layer encoders

Analyze:

- convergence,
- expressiveness,
- overfitting,
- attention structure.

Deliverables

1. Source Code

Submit:

- dataset generation scripts,
- baseline implementations,
- transformer implementation,
- training scripts,
- evaluation/inference code.

Your code should be:

- clean,
- reproducible,
- properly structured,
- and easy to run.

---

2. Experimental Report

Submit a short report containing:

- methodology,
- architectural choices,
- numerical representation strategies,
- experiments performed,
- ablation studies,
- evaluation metrics,
- attention visualizations,
- conclusions and observations.

Focus on analysis and reasoning, not just metrics.

---

3. Trained Model + Demo

Submit (mandatorily):

- trained checkpoints,
- inference notebook.

Suggested Demo Ideas

- User inputs 10 numbers
- Model predicts ranks
- Attention heatmap is visualized alongside predictions

Optional:

- interactive UI,
- web demo,
- visualization dashboard.

Suggested Metrics

You may use:

Training Metrics

- Cross-Entropy Loss
- Validation Loss

Evaluation Metrics

- Token-level Accuracy
- Exact Sequence Accuracy
- Out-of-Distribution Accuracy

Recommended Focus Areas

The most important aspect of this assignment is understanding:

- how transformers reason globally,
- how attention performs comparisons,
- how representation affects learning,
- and how architectural choices influence behavior.

Visualization and interpretation are strongly encouraged.

Do not treat this as only a benchmarking task.

Expected Outcomes

By the end of this assignment, you should be comfortable with:

- implementing transformer encoders from scratch,
- understanding attention mathematically,
- representing numerical data in transformers,
- analyzing attention patterns,
- designing ablation studies,
- and evaluating model generalization behavior.

2. Code Refinement

Subtask 2 - Code Repair using Sequence-to-Sequence Models

Overview

In this task, you will explore sequence-to-sequence (seq2seq) modeling through automated code repair. Given a buggy code snippet as input, the model must generate a corrected version of the code.

The goal of this assignment is **not** to achieve the highest benchmark score using large pretrained models. Instead, the focus is on:

- understanding architectural design choices,
- experimenting with different seq2seq approaches,
- and analyzing the strengths and limitations of each approach.

This task is intentionally open-ended and based on experimentation.

Dataset

We will use the CodeXGLUE Code Refinement dataset.
https://huggingface.co/datasets/google/code_x_glue_cc_code_refinement

The dataset contains pairs of buggy code snippets and corrected code snippets.

Input

```
public java.lang.String METHOD_1 ( ) {
   return new TYPE_1 ( STRING_1 ) . format (
      VAR_1 [ ( ( VAR_1 . length ) - 1 ) ] . getTime ( )
   ) ;
}
```

Output

```
public java.lang.String METHOD_1 ( ) {
   return new TYPE_1 ( STRING_1 ) . format (
```

```
        VAR_1 [ ( ( type ) - 1 ) ] . getTime ( )
    ) ;
}
```

Suggested Workflow

The following is only a rough guideline. You are encouraged to explore beyond this.

Build a Baseline

Start with a simple encoder-decoder model like RNN / LSTM / GRU encoder-decoder. Establish a baseline before moving to more advanced architectures.

Improve the Architecture

Experiment with:

- bidirectional encoders,
- deeper recurrent networks,
- different embedding dimensions,
- attention mechanisms,
- dropout,
- gradient clipping,
- different tokenization strategies.

Analyze which changes help and why.

Transformer-based Models

Implement transformer components from scratch.

This includes:

- self-attention,
- multi-head attention,
- positional encoding,
- encoder-decoder attention,

- masking mechanisms.

You may use PyTorch/TensorFlow primitives, but transformer blocks and attention mechanisms should be implemented by you and not imported from high-level libraries.

The purpose is to understand the architecture well.

Ablations and Experiments

A major component of this assignment is experimentation.

You are expected to conduct meaningful ablation studies.

Possible axes of experimentation include:

Architecture

- RNN vs LSTM vs GRU
- attention vs no attention
- recurrent vs transformer architectures
- encoder depth
- decoder depth

Tokenization

- character-level
- whitespace tokenization
- subword tokenization
- vocabulary size effects

Training

- teacher forcing ratio
- learning rate
- batch size
- optimizer choice
- scheduled sampling

Decoding

- greedy decoding
- beam search
- top-k sampling

Representation

- positional encoding variants
- embedding size
- hidden dimension size

Data

- training on shorter sequences only
- robustness to unseen sequence lengths
- multiple bug corruption

Try to visualize and interpret your results rather than only reporting metrics.

Deliverables

1. Source Code

Submit:

- training scripts,
- model implementations,
- preprocessing code,
- evaluation code.
- Notebook with demo

Your repository/notebook should be clean and reproducible.

---

2. Experimental Report

A short report containing:

- methodology,

- architectural choices,
- experiments conducted,
- ablation studies,
- evaluation metrics,
- qualitative outputs,

---

3. Trained Model + Demo

Submit:

- trained checkpoints,
- inference notebook/script,
- optional Streamlit/Gradio demo.

Suggested demo ideas:

- buggy code → repaired code,
- side-by-side comparison of different models,
- attention visualization.

Evaluation Criteria

Your submission will primarily be evaluated on:

- quality of experimentation,
- depth of analysis,
- clarity of reasoning,
- ablation studies,

A smaller but well-analyzed model is preferred over a large model where you cannot explain your decisions.

Suggested Metrics

You may use:

- exact match accuracy,
- BLEU score,
- edit distance,
- token-level accuracy,
- syntax validity.

---

3. Poetry Generation

Subtask 3 - Decoder-Only Transformers for Poetry Generation

Overview

In this task, you will explore autoregressive sequence modeling through poetry generation using decoder-only transformers (GPT-style architectures).

The focus of the subtask is on:

- understanding how different sequence models generate text,
- experimenting with architectural design choices,
- studying the effect of attention in sequence modeling,
- and analyzing the strengths and limitations of different approaches.

Dataset

We will use a poetry corpus for this assignment:

Poetry Dataset

Suggested Workflow

The following is only a rough guideline. You are strongly encouraged to explore beyond this.

1. Build Baselines

Start with simpler autoregressive language models before moving to transformers.

Possible baselines include:

- Bigram models

- MLP-based language models
- RNNs
- LSTMs

Establish baseline generation quality and compare:

- coherence,
- training behavior,
- stylistic consistency,
- and limitations of sequential architectures.

2. Decoder-Only Transformer

Implement and train a GPT-style decoder-only transformer from scratch.

This includes:

- causal self-attention,
- positional encodings,
- masking mechanisms,
- transformer decoder blocks,
- autoregressive training/inference.

You may use PyTorch/TensorFlow primitives, but transformer blocks and attention mechanisms should be implemented by you and not imported from high-level libraries.

The purpose is to understand the architecture deeply rather than simply using pretrained APIs.

3. Inference & Decoding Experiments

Experiment with different decoding strategies during inference.

Possible directions:

- greedy decoding
- temperature sampling
- top-k sampling
- top-p sampling
- repetition penalties
- prompt conditioning

Analyze how decoding strategies affect:

- creativity,
- repetition,

- coherence,
- and stylistic quality.

Ablations and Experiments

An optional component of this assignment is experimentation and analysis.

You are expected to conduct meaningful ablation studies and architectural comparisons.

Possible axes of experimentation include:

Architecture

- Bigram vs MLP vs RNN vs LSTM vs Transformer
- shallow vs deep transformers
- attention vs no-attention models
- adding attention mechanisms to recurrent architectures
- decoder depth
- residual connections
- normalization variants

Tokenization

- character-level tokenization
- word-level tokenization
- subword/BPE tokenization
- vocabulary size effects

Training

- learning rate
- batch size
- optimizer choice
- dropout
- gradient clipping
- context length
- training stability

Decoding

- greedy decoding
- top-k sampling
- top-p sampling
- temperature effects
- repetition penalties

Representation & Attention

- positional encoding variants
- attention visualization
- head specialization
- long-range dependency behavior

Data

- training on different poetry styles
- robustness to longer prompts
- mixing poetry datasets/styles
- stylistic conditioning

Try to visualize and interpret your results rather than only reporting metrics.

Deliverables

1. Source Code

Submit:

- training scripts,
- model implementations,
- preprocessing code,
- evaluation/inference code.

Your repository/notebook should be clean and reproducible.

2. Experimental Report

A short report containing:

- methodology,
- architectural choices,

- experiments conducted,
- ablation studies,
- observations,
- evaluation metrics,
- qualitative outputs,
- and analysis of model behavior.

3. Trained Model + Demo

Submit:

- trained checkpoints,
- inference notebook with a demo, results and any plots,
- optional Streamlit/Gradio demo.

Suggested demo ideas:

- poetry generation interface,
- side-by-side comparison of different models,
- attention visualization,
- decoding strategy comparison.

Suggested Metrics

You may use:

- validation loss
- perplexity
- token-level accuracy
- generation diversity
- repetition statistics
- qualitative evaluation of outputs